

Álgebra Lineal con Aplicaciones (Práctica)

Vectores Propios en el Algoritmo de Jerarquización de Google (PageRank)

El algoritmo PageRank, utilizado inicialmente por Google para ordenar los resultados de búsqueda, es un modelo matemático basado en la teoría de grafos y el álgebra lineal. Este algoritmo calcula la importancia de las páginas web basándose en las conexiones (enlaces) entre ellas, representando las relaciones de enlace mediante una matriz estocástica. Los vectores propios de esta matriz proporcionan el ranking de las páginas web.

La práctica tiene como objetivo ayudar a los estudiantes a entender cómo se aplica el concepto de vectores propios y métodos iterativos para resolver problemas de jerarquización o ranking. Además, se utilizará Python para implementar el algoritmo y simular el proceso de cálculo del PageRank de un conjunto pequeño de páginas.

Objetivo General

Aplicar el concepto de **vectores propios** en el contexto del **algoritmo PageRank**, utilizando software de programación (Python) para calcular el ranking de páginas web en una red de enlaces.

Objetivos Específicos

1. Representar un conjunto de páginas web como un grafo dirigido.
2. Construir la **matriz de enlaces** (hiperligada) a partir del grafo.
3. Implementar el algoritmo **PageRank** usando el concepto de **vectores propios**.
4. Aplicar el **método de potencias** para aproximar el vector propio dominante.
5. Interpretar los resultados del PageRank y su relación con la estructura del grafo.
6. Utilizar Python para realizar los cálculos de matrices y ejecutar el algoritmo de PageRank.

Conocimientos Previos Requeridos

- Operaciones básicas con matrices y vectores.

- Concepto de **vectores propios** y **autovalores**.
- Métodos numéricos, especialmente el **método de potencias**.
- Programación básica en **Python** y manejo de bibliotecas como **NumPy**.

Fundamentos Teóricos

En el modelo de **PageRank**, cada página web se representa como un **nodo** en un **grafo dirigido**. Los enlaces entre páginas se representan como **aristas** en el grafo. La importancia de una página se calcula basándose en las páginas que la enlazan y la importancia de esas páginas.

Matriz Estocástica de Enlaces

El grafo dirigido se puede representar mediante una **matriz de enlaces** H , donde cada entrada H_{ij} indica la proporción de la importancia de la página j que se distribuye a la página i . En esta matriz:

- Las columnas deben sumar 1, ya que cada página distribuye toda su importancia.
- Si una página j no tiene enlaces, se distribuye su importancia uniformemente entre todas las páginas.

Google Matrix y Factor de Damping

Para evitar problemas como los "nodos colapsados" (páginas sin enlaces de salida), se introduce el **factor de damping** d , el cual representa la probabilidad de que el surfista aleatorio haga un salto a una página aleatoria (en lugar de seguir un enlace). La **matriz de Google** G se calcula como:

$$G = dH + (1 - d) \cdot \frac{1}{n} \cdot ee^T$$

donde e es un vector columna de unos, y n es el número de páginas. El valor de d suele ser 0.85.

Instrucciones

Se trabajará con un grafo de ejemplo con 4 páginas:

- $P_1 \rightarrow P_2, P_3$
- $P_2 \rightarrow P_3$
- $P_3 \rightarrow P_1$

- $P_4 \rightarrow P_1, P_3$

A partir de este grafo, los estudiantes construirán la **matriz de enlaces** H .

Construcción de la matriz de enlaces H

Los estudiantes deben construir la matriz de enlaces a partir del grafo dirigido:

$$H = \begin{bmatrix} 0 & 0 & 0.5 & 0.5 \\ 0.5 & 0 & 0 & 0 \\ 0.5 & 1 & 0 & 0.5 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Construcción de la matriz de Google G

Los estudiantes implementarán el cálculo de la **matriz de Google** G con un valor de damping de $d = 0.85$. Esta etapa ayudará a entender cómo el **factor de damping** modifica la matriz de enlaces.

Utilizando el valor de $d = 0.85$, se debe calcular la matriz G :

$$G = 0.85H + 0.15 \cdot \frac{1}{4} \cdot ee^T$$

Implementación del algoritmo PageRank en Python

Los estudiantes implementarán el **método de potencias** para encontrar el **vector propio dominante** de la matriz G . Este vector representará el ranking de las páginas.

Los estudiantes implementarán el siguiente código en Python para calcular el vector **PageRank**:

```
---
```

```
import numpy as np
```

```
# Matriz de enlaces H
```

```
H = np.array([
```

```
    [0.0, 0.0, 0.5, 0.5], # Enlaces de P1
```

```
    [0.5, 0.0, 0.0, 0.0], # Enlaces de P2
```

```
    [0.5, 1.0, 0.0, 0.5], # Enlaces de P3
```

```

    [0.0, 0.0, 0.0, 0.0] # Enlaces de P4
])

# Número de páginas
n = H.shape[0]

# Factor de damping
d = 0.85

# Matriz de Google
e = np.ones((n, 1)) # Vector de unos
G = d * H + (1 - d) * (1 / n) * np.dot(e, e.T)

# Vector de inicio
x = np.ones(n) / n

# Método de potencias
tol = 1e-10
max_iter = 100
for i in range(max_iter):
    x_new = np.dot(G, x)
    if np.linalg.norm(x_new - x, ord=1) < tol:
        break
    x = x_new

# Resultados
print("Vector PageRank final:")

```

```
print(x)
```

```
# Clasificación de páginas
```

```
pages = ['P1', 'P2', 'P3', 'P4']
```

```
ranking = sorted(zip(pages, x), key=lambda t: t[1], reverse=True)
```

```
print("\nRanking de páginas:")
```

```
for page, score in ranking:
```

```
    print(f'{page}: {score:.6f}')
```

```
---
```

Una vez calculado el **PageRank**, los estudiantes analizarán los resultados, observando cuál es la página con mayor importancia y cómo se distribuye la importancia entre las páginas.

Entregables

Los estudiantes deberán entregar:

- El código Python usado para calcular el PageRank.
- La matriz de enlaces H y la matriz de Google G .
- El ranking final de las páginas.
- Un informe que incluya los resultados y una interpretación detallada del ranking.